Project Guide: Biomedical Signal Processing Platform

1. Project Overview

This is a focused biomedical signal processing platform developed by Dr. Ke Yuan's lab at Boston Children's Hospital, specializing in real-time biosignal acquisition, analysis, and mobile health applications. The project consists of three tightly integrated components for EMG/IMU data collection and analysis.

Key Technologies

- **Languages**: Python, Java/Kotlin (Android), C++ (Arduino/ESP32)
- **Frameworks**:
  - Android BLE (Bluetooth Low Energy)
  - Arduino/ESP32 (embedded systems)
  - FreeRTOS (real-time task scheduling)
- **Signal Processing**: NumPy, SciPy, Pandas, Matplotlib
- **Mobile**: Android with FastBLE library
- **Hardware**: Arduino Uno R4 WiFi, ESP32, MAX30105 (SpO2), MAX17048 (Battery), MyoWare EMG sensors

High-Level Architecture

The project consists of three main subsystems:

1. **EMG/IMU Signal Processing System** (`emg_util/`)
   - Offline biosignal analysis from recorded data
   - Fatigue detection and movement analysis
   - Feature extraction for muscle activity monitoring
   - Windowed and swing-aligned analysis modes
   - Rest recommendation system
2. **Android BLE Application** (`btLeApp/`)
   - Mobile app for Bluetooth Low Energy sensor data collection
   - Real-time EMG/IMU/SpO2 visualization
   - On-device signal processing and feature extraction
   - Based on FastBLE library
3. **Arduino/ESP32 Sensor Firmware** (`arduino/`)
   - Embedded firmware for biosensor devices
   - EMG, SpO2, IMU, and battery monitoring
   - BLE communication with efficient binary protocol
   - Multi-sensor data acquisition with FreeRTOS

---

2. Getting Started

Prerequisites

For Python Components:

- Python 3.8+ (3.11 recommended)
- pip package manager
- Virtual environment tool (venv or conda)
- Required libraries: numpy, pandas, scipy, matplotlib, pyyaml

For Android Development:

- Android Studio Arctic Fox or later
- JDK 11+
- Android SDK API 21+ (minimum), API 33+ (target)
- Gradle 7.0+
- Physical Android device (BLE not reliable on emulators)

For Arduino/ESP32 Development:

- Arduino IDE 2.0+ or PlatformIO
- Arduino Uno R4 WiFi board support (for UnoR4_BT)
- ESP32 board support (for myoware_imu_v2)
- Required libraries:
  - ArduinoBLE (for Arduino Uno R4)
  - BLE library (for ESP32)
  - SparkFun MAX1704x Fuel Gauge (for battery monitoring)
  - MAX30105 (for SpO2 sensor)

Installation Instructions

1. Python Environment Setup

```
# Create virtual environment
python -m venv venv

# Activate (Windows)
venv\Scripts\activate

# Activate (Linux/Mac)
source venv/bin/activate

# Install dependencies for EMG processing
cd emg_util
pip install numpy pandas scipy matplotlib scikit-learn pyyaml

# Optional: Install Jupyter for notebooks
pip install jupyter ipykernel
```

2. Android App Setup

```
cd btLeApp

# Build the project
gradlew.bat assembleDebug  # Windows
./gradlew assembleDebug    # Linux/Mac

# Install on connected device
gradlew.bat installDebug   # Windows
./gradlew installDebug     # Linux/Mac
```

3. Arduino Setup

1. Open Arduino IDE
2. Install ArduinoBLE library via Library Manager
3. Select board: Arduino Uno R4 WiFi
4. Open `arduino/UnoR4_BT/UnoR4_BT.ino`
5. Upload to board

Basic Usage Examples

Running EMG Analysis

```
cd emg_util

# Process EMG/IMU logs with windowed features
python emg_util.py --emg-imu "logs/emg_imu_log_*.csv" --emg-fs 2000 --mode
windowed --out features.csv --save-pages output/

# Process with swing-aligned features (for tennis/sports analysis)
python emg_util.py --emg-imu "logs/emg_imu_log_*.csv" --emg-fs 2000 --mode swing -
-save-pages output/
```

Using the Android App

1. Launch the app on your Android device
2. Tap "Scan" to discover BLE devices
3. Connect to your sensor (e.g., "R4_RYANW_EMG")
4. Tap "Start Monitor" to begin real-time data collection
5. View EMG waveforms and extracted features

Running Tests

Currently, the project uses manual testing and validation:

- **EMG Processing**: Use sample CSV files in `emg_util/logs/`

- **Android App**: Use demo mode or connect to Arduino/ESP32 device
- **Arduino/ESP32**: Monitor Serial output at 115200 baud (ESP32) or 9600 baud (Arduino Uno R4)

---

3. Project Structure

Main Directories

```
.
├── emg_util/                    # EMG/IMU signal processing
│   ├── emg_util.py             # Main analysis script (windowed + swing modes)
│   ├── imu_feature.py          # IMU feature extraction (acc, gyro, jerk)
│   ├── feature_ml.py           # ML features (MAV, RMS, WL, ZC, SSC)
│   ├── capture_loader.py       # Multi-file CSV data loading
│   ├── rest_recommendation.py # Fatigue-based rest recommendations
│   ├── emg_util_yaml_runner.py # YAML-based batch processing
│   ├── config.yaml             # Analysis configuration parameters
│   ├── config_fast.yaml        # Fast processing configuration
│   ├── logs/                   # Recorded EMG/IMU data logs
│   ├── out/                    # Analysis outputs and plots
│   ├── emg_pages/              # Paged plot outputs
│   ├── fatigue_analysis.ipynb # Jupyter notebook for fatigue analysis
│   └── tennis_analysis_cloud.ipynb # Sports analysis notebook
│
├── btLeApp/                     # Android BLE application
│   ├── app/                    # Main application module
│   │   ├── src/main/java/com/clj/blesample/
│   │   │   ├── operation/     # BLE operations & UI activities
│   │   │   │   ├── PlotActivity.java      # Real-time plotting
│   │   │   │   ├── EmgView.java           # Custom EMG waveform view
│   │   │   │   └── FeaturePlotView.java   # Feature visualization
│   │   │   └── processing/    # Signal processing
│   │   │       └── EmgProcessor.java      # Pure Java DSP
│   │   ├── src/main/python/   # Python analysis scripts
│   │   │   ├── emg_util_nopandas.py       # Lightweight EMG utils
│   │   │   ├── btcentral_offline_decode.py # Offline data decoder
│   │   │   └── analysis_bridge.py         # Python-Java bridge
│   │   └── src/main/res/      # Android resources
│   ├── FastBleLib/             # BLE library module
│   ├── build.gradle            # Project build configuration
│   ├── settings.gradle         # Multi-module project structure
│   ├── README.md               # FastBLE library documentation
│   ├── INTEGRATION_SUMMARY.md # EMG processor integration details
│   └── UI_UPDATE_COMPLETE.md  # UI implementation notes
│
└── arduino/                     # Arduino/ESP32 firmware and utilities
    ├── UnoR4_BT/               # Arduino Uno R4 BLE EMG sensor
    │   └── UnoR4_BT.ino        # Simple EMG sensor with BLE notify
    ├── myoware_imu_v2/         # ESP32 multi-sensor firmware (MAIN)
    │   └── myoware_imu_v2.ino # EMG+IMU+SpO2+Battery with FreeRTOS
    ├── SPO2/                   # Pulse oximetry sensor firmware
    │   └── SPO2.ino            # SpO2 and heart rate monitoring
```

```
    ├── battery_monitor/       # Battery level monitoring examples
    ├── AD8232_heart_rate_monitor/ # ECG sensor examples
    └── py_scripts/            # Python utilities for Arduino
        ├── btCentral.py       # BLE central device for data collection
        ├── sensor_data.py     # Data logging utilities
        ├── sensordata_plotter.py # Real-time plotting
        ├── ble_find_uuid.py   # BLE device discovery tool
        └── battery_integration_patch.md  # Battery integration guide
```

Removed Folders (No Longer in Project)

The following folders have been removed from the project:

- `voc_datasets_coco/`: Pericyte image analysis system (moved to separate project)
- `image_twin/`: PDF image extraction utility (no longer needed)
- `input/`, `output/`: Sample image directories (removed with image processing)
- `tmp/`: Temporary YOLO datasets (removed with image processing)

Key Files and Their Roles

EMG/IMU ProcessingI

- `emg_util.py`: Comprehensive EMG analysis with windowed and swing-aligned features
- `imu_feature.py`: IMU-specific feature extraction (acceleration, gyroscope, jerk)
- `feature_ml.py`: Machine learning feature extraction (MAV, RMS, WL, ZC, SSC)
- `capture_loader.py`: Multi-file CSV data loading with metadata handling
- `rest_recommendation.py`: Fatigue-based rest recommendation system
- `emg_util_yaml_runner.py`: Batch processing using YAML configuration files

Android Application

- `PlotActivity.java`: Real-time EMG/IMU plotting with feature extraction
- `EmgProcessor.java`: Pure Java signal processing (filters, FFT, wavelets)
- `FeaturePlotView.java`: Custom view for feature visualization
- `EmgView.java`: Custom EMG waveform display widget
- `emg_util_nopandas.py`: Lightweight EMG utilities for Android (no pandas dependency)
- `btcentral_offline_decode.py`: Decode binary BLE data offline
- `analysis_bridge.py`: Bridge between Python analysis and Java UI

Arduino/ESP32 Firmware

- `UnoR4_BT.ino`: Simple EMG sensor with BLE communication (Arduino Uno R4)
- `myoware_imu_v2.ino`: **Main firmware** - EMG+IMU+SpO2+Battery with FreeRTOS (ESP32)
- `SPO2.ino`: Pulse oximetry sensor firmware with heart rate detection
- `battery_monitor/`: Battery level monitoring examples for portable devices
- `AD8232_heart_rate_monitor/`: ECG sensor examples

Python Utilities

- `arduino/py_scripts/btCentral.py`: BLE central device for data collection from Arduino/ESP32
- `arduino/py_scripts/sensor_data.py`: Data logging and file management
- `arduino/py_scripts/ble_find_uuid.py`: BLE device discovery and UUID inspection
- `arduino/py_scripts/sensordata_plotter.py`: Real-time plotting of sensor data
- `arduino/py_scripts/battery_integration_patch.md`: Complete guide for adding battery monitoring

Important Configuration Files

- `emg_util/config.yaml`: EMG analysis parameters (sampling rates, filters, windows, baseline settings)
- `emg_util/config_fast.yaml`: Fast processing configuration for quick analysis
- `btLeApp/build.gradle`: Android build configuration and dependencies
- `btLeApp/settings.gradle`: Multi-module project structure (app + FastBleLib)
- `btLeApp/config.gradle`: Shared configuration across modules

Excluded Folders (in .gitignore)

- `emg_util/.idea/`: IDE configuration files
- `btLeApp/.idea/`, `btLeApp/build/`: Android Studio and build artifacts
- `arduino/libraries/`: Arduino library cache
- `arduino/BT_talk/`: Deprecated Bluetooth tools
- `*/__pycache__/`: Python bytecode cache
- `*.log`, `*.cmd`: Log files and command scripts
- `*.zip`: Archived log files

---

4. Development Workflow

Coding Standards

Python

- Follow PEP 8 style guide
- Use type hints where applicable
- Document functions with docstrings
- Use meaningful variable names (e.g., `emg_fs_hz` not `fs`)
- Prefer NumPy/Pandas operations over loops

Java/Android

- Follow Android coding conventions
- Use camelCase for methods and variables
- Implement proper lifecycle management (onCreate, onDestroy)
- Always cancel timers and close resources
- Use thread-safe operations for shared data

Arduino/C++

- Use descriptive function names

- Comment hardware-specific configurations
- Initialize all variables
- Use `const` for pin definitions

Testing Approach

Manual Testing

1. **Image Processing**: Test with known cell images, verify detection accuracy
2. **EMG Analysis**: Use recorded data, validate feature extraction
3. **Android App**: Test on physical devices with real sensors
4. **Arduino**: Monitor serial output, verify BLE communication

Validation

- Compare processed images with ground truth annotations
- Verify EMG features against published literature values
- Check BLE data integrity with packet counters
- Validate signal processing with synthetic test signals

Build and Deployment Process

Python Web Service

```
# Development
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# Production (with gunicorn)
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

Android App

```
# Debug build
./gradlew assembleDebug

# Release build (requires signing)
./gradlew assembleRelease

# Install on device
./gradlew installDebug
adb logcat -s EMG EMG_Features  # View logs
```

Arduino

1. Verify board and port selection
2. Compile and upload via Arduino IDE

3. Monitor serial output for debugging

Contribution Guidelines

1. **Branch Strategy**: Create feature branches from main
2. **Commit Messages**: Use descriptive messages (e.g., "Fix EMG plotting bug in EmgView")
3. **Code Review**: Test thoroughly before committing
4. **Documentation**: Update relevant README files
5. **Dependencies**: Document any new library requirements

---

5. Key Concepts

Domain-Specific Terminology

Biomedical Signals

- **EMG (Electromyography)**: Electrical activity of muscles
- **IMU (Inertial Measurement Unit)**: Accelerometer + gyroscope sensor
- **SpO2**: Blood oxygen saturation percentage (pulse oximetry)
- **HR (Heart Rate)**: Beats per minute
- **PPG (Photoplethysmography)**: Optical measurement of blood volume changes

Signal Processing

- **RMS (Root Mean Square)**: Measure of signal amplitude/power
- **MNF (Mean Frequency)**: Average frequency of power spectrum
- **MDF (Median Frequency)**: Frequency dividing spectrum into equal power halves
- **Envelope**: Smooth curve outlining signal extremes
- **Bandpass Filter**: Allows frequencies within a range (e.g., 20-450 Hz)
- **Notch Filter**: Removes specific frequency (e.g., 60 Hz power line noise)
- **MAV (Mean Absolute Value)**: Average of absolute signal values
- **WL (Waveform Length)**: Sum of absolute differences between adjacent samples
- **ZC (Zero Crossings)**: Number of times signal crosses zero
- **SSC (Slope Sign Changes)**: Number of times slope changes direction

Fatigue Analysis

- **Baseline**: Reference values from early, non-fatigued period
- **MDF Drop**: Decrease in median frequency indicating muscle fatigue
- **Swing Window**: Time segment aligned to movement events (e.g., tennis swing)
- **Windowed Analysis**: Fixed-duration overlapping windows for feature extraction
- **Swing-Aligned Analysis**: Event-triggered windows based on IMU movement detection

BLE (Bluetooth Low Energy)

- **Central**: Device that initiates connection (e.g., smartphone)
- **Peripheral**: Device that advertises and accepts connections (e.g., sensor)
- **Characteristic**: Data endpoint for reading/writing

- **Notify**: Push data from peripheral to central
- **MTU (Maximum Transmission Unit)**: Maximum packet size for BLE communication
- **UUID**: Unique identifier for BLE services and characteristics

Hardware Components

- **MyoWare**: EMG sensor module for muscle activity measurement
- **MAX30105**: Optical sensor for SpO2 and heart rate
- **MAX17048**: LiPo battery fuel gauge for monitoring battery level
- **MPU6050/ICM-20948**: IMU sensors with accelerometer and gyroscope

Core Abstractions

Signal Processing Pipeline

```
Raw Data → Detrend → Bandpass Filter → Notch Filter → Feature Extraction →
Analysis
```

BLE Data Flow

```
Arduino/ESP32 Sensor → BLE Notify → Android App → Buffer → Processing →
Display/Save
```

Offline Analysis Pipeline

```
Recorded CSV → Load Data → Preprocess → Extract Features → Detect Fatigue →
Generate Reports
```

Multi-Sensor Data Acquisition (ESP32)

```
FreeRTOS Task 1: EMG Sampling (2000 Hz)
FreeRTOS Task 2: IMU Sampling (200 Hz)
FreeRTOS Task 3: SpO2 Processing (25 Hz)
Main Loop: BLE Communication + Battery Monitoring
```

Design Patterns Used

1. **Pipeline Pattern**: Sequential data transformations (image processing, signal processing)
2. **Observer Pattern**: BLE callbacks, progress reporting (SSE)
3. **Factory Pattern**: Feature extractor creation based on configuration
4. **Singleton Pattern**: BleManager in Android app

5. **Strategy Pattern**: Different baseline selection strategies (swing vs. window)

---

6. Common Tasks

Task 1: Add a New EMG Feature

1. **Define the feature function** in `emg_util.py`:

```
def feat_new_metric(sig: np.ndarray) -> float:
    """Compute new metric from signal."""
    return float(np.nanmean(np.abs(np.diff(sig))))
```

2. **Add to feature extraction** in `compute_emg_window_features()`:

```
rows.append({
    # ... existing features ...
    "new_metric": feat_new_metric(seg),
})
```

3. **Update baseline computation** in `compute_baseline_from_windows()`:

```
for k in ['mdf','rms','mav','wl','new_metric']:  # Add 'new_metric'
    if k in seg.columns:
        base[k] = float(np.nanmedian(seg[k].to_numpy(dtype=float)))
```

Task 2: Collect Data from Arduino/ESP32 Sensor

1. **Upload firmware**: Flash `myoware_imu_v2.ino` to ESP32 (or `UnoR4_BT.ino` to Arduino)
2. **Run Python script**:

```
cd arduino/py_scripts
python btCentral.py --device "MyoWare_IMU" --duration 60
```

3. **Data saved to**: `logs/emg_imu_log_TIMESTAMP.csv` and `logs/vitals_log_TIMESTAMP.csv`

Task 3: Analyze Recorded EMG Data

```
cd emg_util

# Basic windowed analysis
python emg_util.py \
  --emg-imu "logs/emg_imu_log_*.csv" \
```

```
  --emg-fs 2000 \
  --mode windowed \
  --win 1.0 \
  --step 0.25 \
  --out features.csv \
  --save-pages output/

# Swing-aligned analysis (for sports)
python emg_util.py \
  --emg-imu "logs/emg_imu_log_*.csv" \
  --emg-fs 2000 \
  --mode swing \
  --swing-z 5.0 \
  --swing-min-gap-ms 300 \
  --save-pages output/
```

Task 4: Add Battery Monitoring to ESP32 Firmware

1. **Install MAX17048 library**: Arduino IDE → Library Manager → "SparkFun MAX1704x"
2. **Follow the integration guide**: `arduino/py_scripts/battery_integration_patch.md`
3. **Key steps**:
   - Add library include and global objects
   - Initialize MAX17048 in `setup()` with graceful degradation
   - Add battery reading function with safety checks
   - Update `VitalsState` struct to include battery fields
   - Modify `send_vitals_frame()` to include battery data
4. **Update Python decoder**: Modify `btCentral.py` to parse battery fields

Task 5: Debug Android BLE Connection

1. **Enable Bluetooth debugging**:

```
adb logcat -s EMG EMG_Features BLE BluetoothGatt
```

2. **Check device advertising**:

```
cd arduino/py_scripts
python ble_find_uuid.py
```

3. **Verify characteristics**: Ensure UUIDs match between Arduino/ESP32 and Android app

4. **Test with Python first**: Use `btCentral.py` to verify sensor is working

Task 6: Configure EMG Analysis Parameters

Edit `emg_util/config.yaml`:

```yaml
# Sampling rates
emg_fs_hz: 2000
imu_fs_hz: 200

# Filtering
bandpass_low_hz: 20
bandpass_high_hz: 450
notch_freq_hz: 60

# Windowing
window_sec: 1.0
step_sec: 0.25

# Baseline
baseline_mode: "swing"  # or "window"
warmup_skip_min: 5.0
baseline_swings: 20

# Fatigue detection
fatigue_mdf_drop_threshold_pct: 15.0
```

Then run with YAML:

```
python emg_util_yaml_runner.py --config config.yaml
```

---

7. Troubleshooting

Common Issues and Solutions

Issue: "No EMG rows found" error

Cause: CSV file doesn't contain expected columns or data format is incorrect
Solution:

- Verify CSV has columns: `ts_ms, env, raw, ref, ax, ay, az, gx, gy, gz`
- Check for empty or corrupted files
- Use `capture_loader.py` to validate data format

Issue: Android app shows straight lines instead of EMG waveform

Cause: Bug in EmgView.java (already fixed in current version)
Solution: Ensure you have the latest version with corrected integer division:

```
startY0 = mHeight / 4;        // Not: mHeight * (1 / 4)
startY1 = mHeight * 3 / 4;    // Not: mHeight & (3 / 4)
```

Issue: BLE connection fails or drops frequently

Cause: Distance, interference, or power management
Solution:

- Keep devices within 5 meters
- Disable battery optimization for the app (Android Settings → Apps → Battery)
- Check Arduino/ESP32 serial monitor for errors
- Verify BLE service UUIDs match between firmware and app
- Ensure MTU negotiation succeeds (check logs for "MTU=" value)

Issue: "Not enough data for processing" in Android app

Cause: Buffer hasn't filled yet (needs 500+ samples)
Solution: Wait 1-2 seconds after starting. This is normal behavior.

Issue: NaN values in extracted features

Cause: Signal too short, all zeros, or insufficient data
Solution:

- Check data is arriving correctly (not all zeros)
- Increase window size
- Verify sampling rate is correct
- Check for sensor disconnection

Issue: ESP32 firmware crashes or resets

Cause: Stack overflow, memory exhaustion, or watchdog timeout
Solution:

- Check Serial Monitor for crash logs
- Increase task stack sizes in FreeRTOS configuration
- Reduce buffer sizes if memory is low
- Ensure watchdog is fed in long-running loops

Issue: Battery monitoring not working

Cause: MAX17048 not connected or I2C address conflict
Solution:

- Verify MAX17048 is connected to correct I2C pins
- Check I2C address (default 0x36)
- Run I2C scanner sketch to detect devices
- System should work without battery monitor (graceful degradation)

Issue: SpO2 readings are invalid (0xFFFF)

Cause: MAX30105 sensor not detecting finger or poor contact
Solution:

- Ensure finger is placed firmly on sensor
- Check LED brightness setting
- Verify MAX30105 is properly connected
- Wait 5-10 seconds for readings to stabilize

Debugging Tips

Python

```
# Enable verbose logging
python emg_util.py --emg-imu "logs/*.csv" --emg-fs 2000 --mode windowed -v

# Use IPython for interactive debugging
ipython
%run emg_util.py --emg-imu "logs/*.csv" --emg-fs 2000

# Check CSV file structure
python -c "import pandas as pd;
print(pd.read_csv('logs/emg_imu_log_xxx.csv').head())"
```

Android

```
# Filter logs by tag
adb logcat -s EMG EMG_Features BLE

# Clear logs before testing
adb logcat -c

# Save logs to file
adb logcat > debug.log

# Check for crashes
adb logcat | grep -i "fatal\|exception\|crash"
```

Arduino/ESP32

```
// Add debug prints
Serial.print("EMG value: ");
Serial.println(emgData);

// Monitor at correct baud rate
// Arduino Uno R4: 9600 baud
// ESP32: 115200 baud
```

```
// Check FreeRTOS task status (ESP32)
vTaskList(taskListBuffer);
Serial.println(taskListBuffer);

// Check heap memory (ESP32)
Serial.printf("Free heap: %d bytes\n", ESP.getFreeHeap());
```

BLE Debugging

```
# Discover BLE devices
cd arduino/py_scripts
python ble_find_uuid.py

# Test connection with verbose output
python btCentral.py --device "MyoWare_IMU" --duration 10 --verbose

# Check BLE packet structure
python btcentral_offline_decode.py logs/emg_imu_log_xxx.csv
```

---

8. References

Documentation

- **FastBLE Library**: https://github.com/Jasonchenlijian/FastBle
- **ArduinoBLE**: https://www.arduino.cc/reference/en/libraries/arduinoble/
- **ESP32 BLE**: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/bluetooth/index.html
- **FreeRTOS**: https://www.freertos.org/Documentation/RTOS_book.html
- **SciPy Signal Processing**: https://docs.scipy.org/doc/scipy/reference/signal.html
- **MAX30105 SpO2 Sensor**: https://www.maximintegrated.com/en/products/interface/sensor-interface/MAX30105.html
- **MAX17048 Fuel Gauge**: https://www.maximintegrated.com/en/products/power/battery-management/MAX17048.html

Research Papers & Methods

- EMG signal processing and fatigue detection methodologies
- Bluetooth Low Energy for biosignal acquisition
- Real-time embedded signal processing on resource-constrained devices
- Multi-modal sensor fusion (EMG + IMU + physiological signals)

Internal Resources

- **btLeApp/README.md**: FastBLE library usage and examples
- **btLeApp/INTEGRATION_SUMMARY.md**: Android EMG processor integration details

- **btLeApp/UI_UPDATE_COMPLETE.md**: UI implementation notes
- **arduino/py_scripts/battery_integration_patch.md**: Complete battery monitoring integration guide

Key Algorithms

- **Butterworth Filter**: Bandpass and notch filtering for EMG
- **Welch's Method**: Power spectral density estimation for MNF/MDF
- **Wavelet Transform**: Signal decomposition (db4 wavelet)
- **Sample Entropy**: Signal regularity and complexity measure
- **Z-score Movement Detection**: IMU-based swing detection without FSR

Contact

- **Lab**: Dr. Ke Yuan Lab, Boston Children's Hospital
- **Project Focus**: Biomedical signal processing for wearable health monitoring

---

Notes for Developers

Performance Considerations

- **EMG Processing**: Use NumPy vectorized operations; avoid Python loops
- **Android**: Run heavy processing in background threads; update UI on main thread
- **BLE**: Limit notification rate to avoid buffer overflow (max ~100 Hz recommended)
- **ESP32**: Use FreeRTOS tasks for concurrent sensor sampling; monitor stack usage
- **Battery Life**: Reduce sampling rates, use sleep modes, optimize BLE connection intervals

Security Considerations

- **BLE**: No authentication in current implementation; add pairing for sensitive data
- **Data Privacy**: EMG/IMU data can reveal personal health information
- **File Storage**: Ensure proper permissions on log files

Future Enhancements

- [ ] Add automated unit tests for signal processing functions
- [ ] Implement real-time fatigue alerts in Android app
- [ ] Add gesture recognition using EMG features
- [ ] Support for additional sensor types (ECG, EEG, GSR)
- [ ] Cloud storage integration for data backup
- [ ] Multi-user support with authentication
- [ ] Export analysis reports as PDF
- [ ] Machine learning models for activity classification
- [ ] Real-time coaching/feedback based on fatigue detection

Version History

- **v1.3** (Current): Simplified project structure, removed image processing, enhanced ESP32 firmware
- **v1.2**: Multi-channel overlay options, improved fatigue detection, swing-aligned analysis
- **v1.1**: Swing-aligned feature extraction, IMU movement detection, Android EMG processor

- **v1.0**: Initial release with basic EMG/IMU processing

Project Organization Notes

- **Active Development**: Focus on `emg_util/`, `btLeApp/`, and `arduino/myoware_imu_v2/`
- **Main Firmware**: `arduino/myoware_imu_v2/myoware_imu_v2.ino` (ESP32 with FreeRTOS)
- **Legacy Firmware**: `arduino/UnoR4_BT/` (simpler Arduino Uno R4 version)
- **Archived Components**: Older Arduino sketches kept for reference but not actively maintained
- **Data Logs**: `emg_util/logs/` contains recorded EMG/IMU sessions with metadata JSON files
- **Analysis Outputs**: `emg_util/out/` and `emg_util/emg_pages/` contain generated plots and CSV files

---

9. Making Single-Subject Research Novel and Publishable

Challenge: Limited to Self-Data Collection

When you can only collect sensor data from yourself, traditional clinical studies with large sample sizes aren't feasible. However, there are several strategies to make your research novel and publishable:

Strategy 1: Focus on Methodology and Technology Innovation

Novel Contributions:

- **System Design**: Document your complete hardware/software pipeline
    - Multi-sensor integration (EMG + IMU + SpO2 + Battery)
    - Real-time processing on resource-constrained devices
    - Efficient BLE protocol design
    - Open-source, reproducible system
- **Algorithm Development**: Create new analysis methods
    - Swing-aligned feature extraction without FSR sensors
    - Robust z-score movement detection from IMU alone
    - Real-time fatigue detection algorithms
    - Multi-modal sensor fusion techniques
- **Validation Approach**: Use yourself as a "test bed"
    - Demonstrate repeatability across multiple sessions
    - Show consistency under controlled conditions
    - Compare different algorithms on the same dataset

Publication Venues:

- IEEE Sensors Journal
- Sensors (MDPI)
- IEEE Transactions on Biomedical Engineering (Methods papers)
- Journal of Open Source Software (for software contributions)

Strategy 2: N-of-1 Study Design (Single-Subject Research)

Rigorous Single-Subject Methodology:

- **Multiple Baseline Measurements**: Collect extensive baseline data over weeks/months
- **Repeated Measures**: Perform the same task/protocol multiple times
- **Within-Subject Comparisons**: Compare different conditions on yourself
  - Different exercise intensities
  - Different times of day
  - With/without interventions (rest, nutrition, etc.)
- **Time-Series Analysis**: Use statistical methods designed for longitudinal data
  - Autoregressive models
  - Change-point detection
  - Trend analysis

Example Study Designs:

1. **Fatigue Progression Study**: Track your muscle fatigue over 50+ tennis sessions
2. **Intervention Study**: Compare fatigue with/without rest breaks (ABAB design)
3. **Circadian Rhythm Study**: Measure EMG patterns at different times of day
4. **Training Effect Study**: Track changes in fatigue resistance over 3-6 months

Publication Venues:

- Frontiers in Physiology (Case Reports section)
- BMJ Case Reports
- Journal of Medical Case Reports
- Sports Medicine - Open (Case Studies)

Strategy 3: Proof-of-Concept and Feasibility Studies

Frame Your Work As:

- **Feasibility Study**: "Is it feasible to detect fatigue using wearable sensors?"
- **Proof-of-Concept**: "Can swing-aligned analysis improve fatigue detection?"
- **Pilot Study**: "Preliminary investigation of multi-modal sensor fusion"
- **Technical Validation**: "Validation of a novel biosensor system"

Key Elements:

- Clearly state this is a proof-of-concept with one subject
- Emphasize the technical innovation
- Discuss how this could scale to larger studies
- Provide detailed methodology for others to replicate
- Share all code and data (open science)

Publication Venues:

- PLOS ONE (accepts pilot studies)
- Scientific Reports (Nature)
- Frontiers journals (lower barrier for pilot studies)
- Conference papers (IEEE EMBC, ACM ICMI, etc.)

Strategy 4: Open Science and Reproducibility Focus

Make Your Contribution About Reproducibility:

- **Open-Source Everything**:
  - Hardware designs (schematics, BOM)
  - Firmware (Arduino/ESP32 code)
  - Mobile app (Android source)
  - Analysis scripts (Python)
  - Raw data (anonymized if needed)
- **Detailed Documentation**:
  - Step-by-step build instructions
  - Calibration procedures
  - Data collection protocols
  - Analysis pipelines
- **Reproducibility Package**:
  - Docker containers for analysis
  - Jupyter notebooks with all figures
  - Automated analysis scripts
  - Unit tests for algorithms

Publication Venues:

- Journal of Open Source Software (JOSS)
- SoftwareX (Elsevier)
- HardwareX (Elsevier) - for hardware designs
- F1000Research (with open peer review)

Strategy 5: Comparative Analysis and Benchmarking

Compare Multiple Approaches on Your Data:

- **Algorithm Comparison**: Test 5-10 different fatigue detection algorithms
- **Feature Comparison**: Compare time-domain vs. frequency-domain features
- **Sensor Comparison**: EMG alone vs. EMG+IMU vs. EMG+IMU+SpO2
- **Window Comparison**: Fixed windows vs. swing-aligned windows

Novel Contribution: Comprehensive benchmarking study

- "Which features are most sensitive to fatigue?"
- "What is the optimal window size for fatigue detection?"
- "How much does IMU data improve fatigue detection?"

Publication Venues:

- IEEE Transactions on Neural Systems and Rehabilitation Engineering
- Journal of NeuroEngineering and Rehabilitation

- Biomedical Signal Processing and Control

Strategy 6: Longitudinal Self-Tracking Study

Collect Data Over Extended Period:

- **Duration**: 6-12 months of continuous monitoring
- **Frequency**: Multiple sessions per week
- **Variety**: Different activities, conditions, states

Research Questions:

- How does fatigue change with training?
- Can we predict performance from baseline measurements?
- What are the long-term trends in muscle activity?
- How do external factors (sleep, nutrition) affect fatigue?

Novel Aspects:

- Unprecedented temporal resolution
- Real-world, ecological validity
- Personalized medicine approach
- Digital phenotyping

Publication Venues:

- npj Digital Medicine (Nature)
- Digital Health (SAGE)
- JMIR mHealth and uHealth
- Quantified Self research venues

Strategy 7: Combine with Existing Public Datasets

Augment Your Single-Subject Data:

- Use public EMG datasets for algorithm development
- Validate on public data, then apply to your data
- Show your system works on both public and personal data

Public EMG Datasets:

- Ninapro (hand/arm EMG)
- EMG Physical Action Data Set (UCI)
- Putnam EMG Database (PhysioNet)
- CapgMyo (gesture recognition)

Novel Contribution: Cross-dataset validation

- "Algorithm developed on public data, validated on real-world personal data"

Strategy 8: Focus on Real-World Applications

Practical Use Cases:

- **Personal Health Monitoring**: Self-tracking for injury prevention
- **Athletic Training**: Optimize training load
- **Rehabilitation**: Track recovery progress
- **Occupational Health**: Prevent repetitive strain injuries

Frame as:

- "Real-world deployment of wearable fatigue monitoring"
- "Lessons learned from 6 months of continuous EMG monitoring"
- "Practical considerations for personal biosensor systems"

Publication Venues:

- Wearable Technologies (MDPI)
- Journal of Sports Sciences
- Applied Ergonomics
- Work: A Journal of Prevention, Assessment and Rehabilitation

Key Success Factors

1. **Be Transparent**: Clearly state limitations of single-subject design
2. **Emphasize Rigor**: Use proper statistical methods, multiple sessions, controls
3. **Focus on Innovation**: Highlight what's novel (technology, methods, insights)
4. **Enable Replication**: Provide everything needed for others to reproduce
5. **Tell a Story**: Frame as proof-of-concept, feasibility, or methodology paper
6. **Target Right Venues**: Choose journals/conferences that accept pilot studies
7. **Collaborate**: Partner with clinicians/researchers who have access to more subjects

Recommended Publication Strategy for Your Project

Given your current system, I recommend a **two-paper approach**:

Paper 1: Technology/Methods Paper

- Title: "An Open-Source Multi-Modal Wearable System for Real-Time Muscle Fatigue Monitoring"
- Focus: System design, BLE protocol, real-time processing, open-source contribution
- Venue: Sensors (MDPI) or HardwareX
- Strength: Complete reproducible system with code/hardware

Paper 2: Methodology/Algorithm Paper

- Title: "Swing-Aligned Feature Extraction for Fatigue Detection in Repetitive Movements: A Proof-of-Concept Study"
- Focus: Novel algorithm, comparison with windowed approach, single-subject validation
- Venue: Biomedical Signal Processing and Control or IEEE Sensors Journal
- Strength: Novel algorithm with rigorous validation

Additional Tips

- **Preprint First**: Post on arXiv or bioRxiv before submission
- **Conference Presentations**: Present at IEEE EMBC, ACM conferences for feedback
- **Blog Posts**: Write technical blog posts to build community interest
- **GitHub Stars**: Promote your open-source code to demonstrate impact
- **Collaborations**: Reach out to researchers who might want to use your system

---

Last Updated: February 2026 This guide is maintained by the development team. Please update as the project evolves.